Section 9.2: Zero-Trust Identity, Cryptography & Network Security

> ***Study Guide*** *— 2 files in this series | Section 9.**

Executive Overview

Zero-trust security assumes every request is hostile — no implicit trust based on network location, service identity, or prior authentication. Every access decision must be continuously verified. This guide covers authentication tokens, identity protocols, and secret management with comparative analysis and interview-focused design problems.

---

1. Stateless Auth Tokens vs. Stateful Sessions

JWT (JSON Web Tokens)

Token anatomy:

```
Format: Base64url(Header) . Base64url(Payload) . Signature

Header:
{
  "alg": "RS256",   ← asymmetric: private key signs, public key verifies
  "typ": "JWT"
}

Payload (claims):
{
  "sub":   "user123",       ← subject (who)
  "exp":   1717182000,      ← expiry (Unix timestamp)
  "iat":   1717178400,      ← issued at
  "scope": "read:users write:orders"
}

Signature:
```

```
  RS256: RSA_Sign(privateKey, base64url(header) + "." + base64url(payload))
  HS256: HMAC-SHA256(sharedSecret, header + "." + payload)
```

Why RS256 over HS256 in production:

```
HS256 (symmetric):
  Same secret used to sign AND verify
  Every service that verifies must hold the secret
  Secret compromise → all tokens forgeable

RS256 (asymmetric):
  Auth server holds private key (signs)
  All services hold public key only (verify)
  Public key compromise → can only verify, not forge
  Recommended for multi-service architectures
```

Stateless benefits and limitations:

```
Benefits:
  No database lookup on each request (decode + verify signature)
  Horizontal scaling: no shared session store needed
  Client can read claims (user ID, roles) without server

Limitations:
  Cannot revoke before expiry (no server-side state to invalidate)
  Size: typically 500B–2KB (vs 32-byte session ID)
  Secret key rotation requires re-issuing all tokens
```

Stateful Sessions

Server-side tracking:

```
def create_session(user_id):
    session_id = generate_random_id()   # 256-bit random, opaque to client

    redis.setex(
        f"session:{session_id}",
        user_id,
        ttl=3600                        # 1-hour TTL
    )

    response.set_cookie(
        "session_id",   session_id,
        http_only=True,                 # Not accessible to JavaScript (XSS
protection)
        secure=True,                    # HTTPS only
```

```
        same_site="Strict"              # CSRF protection
    )
```

Instant revocation:

```
# Log user out everywhere, immediately:
redis.delete(f"session:{session_id}")

# Revoke all sessions for a user (password change, compromise):
for session_id in redis.smembers(f"user_sessions:{user_id}"):
    redis.delete(f"session:{session_id}")
```

Trade-Off Matrix

Aspect	JWT (Stateless)	Stateful Sessions
Scalability	Excellent — no shared storage	Requires shared store (Redis)
Revocation	Complex — needs blocklist or short TTL	Instant — delete the record
Token size	500B–2KB	32 bytes (session ID only)
Server overhead	CPU (signature verify)	Network (Redis lookup)
Mobile-friendly	Yes (no cookie required)	Cookie management needed
Microservices	Excellent (each service verifies independently)	Requires session store access
Security incident response	Slow (wait for expiry or blocklist)	Instant (delete session)

Decision heuristic: - High-scale microservices, mobile apps, APIs → **JWT with short TTL (15 min) + refresh tokens** - Web apps needing instant revocation (banking, admin panels) → **Stateful sessions or hybrid** - Never: Long-lived JWTs (hours/days) without a revocation mechanism

---

Interview Design Problem 1: Session Management for 1M Concurrent Users with Instant Logout

Problem: Design a session system for 1M concurrent users that supports instant logout (e.g., when a user's device is stolen). How do you balance scalability and security?

Solution Framework:

```
Core tension:
  Pure JWT → scales well, but can't revoke until expiry
  Pure stateful → instant revocation, but Redis at 1M sessions is a bottleneck

Hybrid architecture (recommended):

  Token structure:
    Access token:  JWT, 15-minute TTL     → stateless, fast
    Refresh token: Opaque, 30-day TTL     → stored server-side, revocable

  Validation flow:
    Each request → verify JWT signature (no DB call)
    Every 15 minutes → exchange refresh token for new access token
      At refresh: check if refresh token is revoked in Redis

  Logout / compromise response:
    Delete refresh token from Redis → user cannot get new access token
    Existing access token lives max 15 minutes → acceptable window
    For zero-tolerance (e.g., admin accounts): store JWT ID (jti) in
    Redis blocklist with TTL matching remaining token lifetime

  Redis sizing at 1M concurrent sessions:
    Per session: session_id (32B) + user_id (8B) + metadata (~50B) ≈ 100B
    1M sessions × 100B = ~100MB in Redis (trivial)
    Redis Cluster: 3 shards handles this easily

  Local caching to reduce Redis load:
    Cache "is token valid?" result locally for 60 seconds
    Trade-off: up to 60-second delay for revocation propagation
    Acceptable for most cases; skip cache for high-risk operations (wire
transfers)

  Failure mode:
    Redis down → fail-open (accept JWT, skip revocation check) for availability
    Alert immediately; monitor for abuse during outage window

Key insight: Store only what you need for revocation — not full session state.
The access token carries all claims; Redis only tracks "is this revoked?"
This minimizes storage and keeps revocation O(1) regardless of scale.
```

---

2. Identity Protocol Topologies

OAuth 2.0

Authorization Code Flow (most secure for web apps):

```
1. Client redirects user to auth server:
   GET /authorize?client_id=app&redirect_uri=...&response_type=code&state=xyz
```

```
2. User authenticates and consents at auth server

3. Auth server redirects back with code:
   GET /callback?code=AUTH_CODE&state=xyz

4. Client exchanges code for tokens (server-to-server, not in browser):
   POST /token { code, client_secret, redirect_uri }
   Response: { access_token, refresh_token, expires_in }

5. Client uses access_token to call APIs

Security property: User's password never touches the client app.
The `state` parameter prevents CSRF on the callback.
```

PKCE (Proof Key for Code Exchange) — required for public clients:

```
Problem: Mobile apps / SPAs can't safely store a client_secret.
         Auth code interception attack is possible.

PKCE solution:
  1. Client generates: code_verifier = 256-bit random string
  2. Client computes:  code_challenge = BASE64URL(SHA256(code_verifier))
  3. Client sends code_challenge in authorization request
  4. Auth server stores code_challenge
  5. Client sends code_verifier in token exchange request
  6. Auth server verifies: SHA256(code_verifier) == stored code_challenge

Attack prevented: Stolen auth code is useless without the code_verifier,
which was never transmitted — only its hash was.
```

OpenID Connect (OIDC)

Identity layer built on OAuth 2.0:

```
OAuth 2.0 answers: "Can this app access this resource?"
OIDC adds:         "Who is the authenticated user?"

ID Token (JWT) claims:
{
  "iss":            "https://auth.example.com",  ← issuer
  "sub":            "user123",                   ← stable user identifier
  "aud":            "client-app",                ← intended recipient
  "exp":            1717182000,
  "iat":            1717178400,
  "email":          "user@example.com",
  "email_verified": true,
  "nonce":          "abc123"                     ← replay attack prevention
}
```

```
The `nonce` ties the ID token to the original auth request,
preventing replay attacks where an old token is reused.
```

SAML (Security Assertion Markup Language)

Enterprise federation pattern:

```
XML-based, older standard (2002), dominant in enterprise SSO.

SAML assertion contains:
  Subject:     Who the user is (NameID)
  Attributes:  Roles, groups, department
  Conditions:  Validity window, intended audience

<saml:Assertion>
  <saml:Subject>
    <saml:NameID>user@corp.com</saml:NameID>
  </saml:Subject>
  <saml:AttributeStatement>
    <saml:Attribute Name="role">
      <saml:AttributeValue>admin</saml:AttributeValue>
    </saml:Attribute>
  </saml:AttributeStatement>
</saml:Assertion>

Signed with XML Digital Signature — verbose but auditable.
```

Comparative Analysis: OAuth 2.0 vs. OIDC vs. SAML

Factor	OAuth 2.0	OIDC	SAML
Primary purpose	Authorization	Authentication + Authorization	Authentication (SSO)
Token format	Opaque or JWT	JWT (ID token)	XML assertion
Protocol age	2012	2014	2002
Mobile-friendly	Yes (PKCE)	Yes	Poor (XML, redirect-heavy)
Enterprise SSO	Limited	Good	Excellent
Complexity	Medium	Medium	High (XML parsing)
Standard adoption	Universal	Growing	Enterprise dominant

Decision heuristic: - New consumer web/mobile app → **OIDC** (built on OAuth 2.0, modern) - Enterprise SSO with Active Directory / Okta / Ping → **SAML** (expected by IT) - API authorization only (no user identity needed) → **OAuth 2.0 client credentials flow** - Migrating from SAML to modern stack → **OIDC** with an IdP bridge (Keycloak, Auth0)

Interview Design Problem 2: Designing SSO for a B2B SaaS with Enterprise Customers

Problem: You're building a B2B SaaS. Enterprise customers (Acme Corp, ACME uses Okta; another uses Azure AD) need SSO. Consumer customers sign up with Google/GitHub. How do you architect the identity layer?

Solution Framework:

```
Multi-identity-provider architecture:

  Consumer customers:
    OIDC with Google, GitHub as social IdPs
    Your auth layer (Keycloak/Auth0) acts as Identity Broker:
      Normalizes all external identities into your internal user model
      Assigns internal user_id regardless of external IdP

  Enterprise customers:
    Option A: SAML 2.0 SP-initiated SSO
      Customer configures their IdP (Okta, Azure AD) to trust your ACS URL
      You receive SAML assertion, map to internal user
      Pro: enterprises expect this; IT teams know SAML
      Con: XML complexity, harder to debug

    Option B: OIDC with enterprise IdPs
      Okta/Azure AD support OIDC; increasingly common
      Pro: simpler token handling
      Con: older enterprises may not have OIDC configured

  Recommended: Support both SAML and OIDC
    Most B2B platforms (Slack, Salesforce, GitHub Enterprise) do this
    Detect per-tenant: if tenant has SAML config → use SAML flow
                       if tenant has OIDC config → use OIDC flow
                       else → username/password or social

  Just-in-time (JIT) provisioning:
    On first SSO login, automatically create the internal user account
    Map SAML attributes (role, department) to your permission model
    Prevents manual user provisioning overhead

  Tenant isolation:
    Each enterprise tenant has its own IdP configuration record
    Login page: user enters email → system routes to correct IdP
    (This is the "email domain routing" pattern)
```

```
Key insight: Abstract all IdPs behind an internal identity broker.
Your application only ever sees internal user IDs and roles,
regardless of whether auth came from SAML, OIDC, or social login.
This decouples your app from IdP specifics.
```

---

3. Modern Secret Management

Why Static Secrets Are Dangerous

```
Static credential lifecycle problem:
  DB password created → stored in .env file → checked into git accidentally
  → leaked via CI log → never rotated → breach happens

Static secret risks:
  1. Long blast radius (used everywhere, forever)
  2. Rotation is disruptive (must update all services simultaneously)
  3. Access audit is difficult (who used this credential, when?)
```

HashiCorp Vault: Centralized Secret Store

Core secret engines:

```
KV (Key-Value):          Static secrets with versioning
Database:                Dynamic DB credentials (generated per-request)
AWS/GCP/Azure:           Dynamic cloud IAM credentials
SSH:                     Short-lived signed SSH certificates
PKI:                     Internal CA, issue/revoke TLS certs
Transit:                 Encryption-as-a-service (don't expose keys)
```

Dynamic secrets — the key innovation:

```
# Old way: static DB password in config
DB_PASSWORD = "super_secret_static_password"

# Vault dynamic credentials:
vault_client = hvac.Client(url=VAULT_ADDR, token=VAULT_TOKEN)

creds = vault_client.database.generate_credentials(
    mount_point='database',
    name='app-role',
    ttl='1h'
)
# Returns: { username: "v-app-abc123", password: "A3f-random-xyz" }
```

```
# This credential ONLY exists for 1 hour, then auto-expires.
# If leaked, the blast radius is bounded by TTL.
```

Access via AppRole (Kubernetes-native auth):

```
Pod auth flow:
  1. Pod presents Kubernetes ServiceAccount JWT to Vault
  2. Vault calls Kubernetes TokenReview API to verify JWT
  3. Vault issues short-lived Vault token to pod
  4. Pod uses Vault token to fetch secrets
  No long-lived credentials stored in pod spec or ConfigMaps.
```

Secret Rotation

Automated rotation lifecycle:

```
Rotation schedule:
  Passwords:    Every 30 days
  API keys:     Every 90 days
  TLS certs:    Every 90 days (automate with cert-manager)

Rotation pattern (zero-downtime):
  1. Generate new credential in Vault
  2. Deploy new credential to 10% of services
  3. Verify no auth failures for 5 minutes
  4. Roll out to remaining services
  5. Invalidate old credential

Dual-write window:
  During rotation, both old AND new credentials are valid
  Prevents race condition where new cred deployed but not yet active
```

Comparative Analysis: Secret Management Approaches

Approach	Security	Ops Complexity	Rotation	Audit
Environment variables (static)	Low	Low	Manual, disruptive	None
Kubernetes Secrets	Medium (base64, not encrypted by default)	Low	Manual	Limited
K8s Secrets + etcd encryption	Medium-High	Medium	Manual	Limited

Approach	Security	Ops Complexity	Rotation	Audit
HashiCorp Vault (dynamic)	High	High	Automated	Full audit log
Cloud-native (AWS Secrets Manager)	High	Low-Medium	Automated	CloudTrail
Sealed Secrets (Bitnami)	Medium-High	Medium	Manual	Git-based

Decision heuristic: - Small team, early-stage → **AWS Secrets Manager / GCP Secret Manager** (managed, low ops) - Multi-cloud or on-prem → **HashiCorp Vault** (full control, more ops work) - Kubernetes-only → **External Secrets Operator** + cloud secret store (bridge pattern) - Never: secrets in source code, unencrypted ConfigMaps, or plain env vars in CI logs

---

Interview Design Problem 3: Securing Database Credentials Across 50 Microservices

Problem: You have 50 microservices, each connecting to a shared PostgreSQL database with the same static password. A security audit flags this. How do you migrate to a secure credential model with zero downtime?

Solution Framework:

```
Problem analysis:
  50 services × 1 shared password = catastrophic blast radius on leak
  Static password = likely never rotated = high risk
  Zero downtime = cannot restart all services at once

Target architecture: Vault + dynamic DB credentials

Migration plan (phased, zero-downtime):

Phase 1 — Setup Vault (no service changes):
  Deploy Vault cluster with HA (3 nodes, Raft storage)
  Configure Vault database secret engine:
    vault write database/config/postgres \
      plugin_name=postgresql-database-plugin \
      connection_url="postgresql://{{username}}:{{password}}@postgres:5432/db" \
      allowed_roles="app-role" \
      username="vault-admin" \
      password="vault-admin-password"
  Create role: TTL=1h, max_TTL=24h, per-service role names

Phase 2 — Migrate services one at a time (canary approach):
```

```
  For each service:
    Add Vault client sidecar (or library)
    On startup: fetch dynamic credential from Vault
    Old static env var removed after validation
    Monitor for DB auth errors before proceeding to next service

Phase 3 — Rotate the vault-admin credential:
  Now that no service uses the old static password, rotate it
  Only vault-admin credential remains (highly restricted)

Database permission model:
  Each service role gets minimum required permissions:
    service-orders:  SELECT, INSERT on orders table only
    service-users:   SELECT, UPDATE on users table only
  Principle of least privilege enforced at the DB layer

Credential renewal:
  Services renew Vault lease at 50% of TTL (30 min for 1h TTL)
  If Vault is unreachable: use cached credential until expiry
  Circuit breaker: if Vault down > TTL, alert on-call

Key insight: The migration is safer when done service-by-service.
Each service independently moves to dynamic credentials.
There's no "flag day" where everything changes at once.
```

---

4. Zero-Trust Network Principles

Core Tenets

```
Traditional perimeter model:
  "Trust everything inside the network"
  VPN = trusted, external = untrusted
  Breach the perimeter → access everything

Zero-trust model:
  "Never trust, always verify"
  1. Verify identity (who are you?)
  2. Verify device health (is your device compliant?)
  3. Verify context (is this access pattern normal?)
  4. Least-privilege access (only what you need)
  5. Assume breach (log everything, detect lateral movement)
```

mTLS (Mutual TLS) for Service-to-Service Auth

```
Standard TLS: client verifies server certificate
mTLS:         server ALSO verifies client certificate
→ Both parties prove identity
```

```
Service mesh implementation (Istio/Linkerd):
  Each pod gets a short-lived X.509 certificate (SPIFFE identity)
  Certificate: "spiffe://cluster.local/ns/prod/sa/orders-service"
  Auth policy: orders-service can only talk to payments-service
  All other connections rejected at the proxy layer

Benefits:
  No secrets to manage in application code
  Zero-trust between pods inside the cluster
  Automatic certificate rotation (every 24h by default in Istio)
```

Summary: Decision Frameworks

Authentication Token Selection

- **JWT (short-lived, 15 min) + refresh tokens**: Microservices, mobile APIs, high scale.
- **Stateful sessions**: Web apps needing instant revocation, admin panels, banking.
- **Hybrid**: JWT for speed + Redis revocation list for security-critical operations.

Identity Protocol Selection

- **OIDC**: New apps, consumer auth, modern enterprise IdPs.
- **SAML**: Legacy enterprise SSO, Active Directory-heavy environments.
- **OAuth 2.0 only**: Service-to-service authorization without user identity.

Secret Management Selection

- **Vault (dynamic secrets)**: High-security, multi-service, multi-cloud environments.
- **Cloud Secrets Manager**: Simpler ops, single-cloud, small-medium teams.
- **Never**: Static credentials in code, ConfigMaps, or shared across services.

Zero-Trust Principles to Always Apply

1. Short-lived credentials (15 min–1 hour) over long-lived ones.
2. Per-service identities — never shared passwords across services.
3. Audit every access — who, what, when, from where.
4. Assume breach — design for containment, not just prevention.